Grado en Inteligencia Artificial · Universidade da Coruña · Curso 2026/27

Práctica 1

Seguimiento de un vehículo en la ciudad de Robobo

Aprendizaje por refuerzo en espacios continuos. Desarrollo en simulación y transferencia al entorno real

Entrega	**Domingo 18 de octubre de 2026**, por el campus online
Formato	Archivo .zip con (1) memoria, (2) declaración de uso de IA, (3) código fuente comentado y (4) evidencias: modelos, registros y vídeos
Memoria	Máximo 6 páginas, más los anexos que hagan falta, en Calibri 11 pt con interlineado sencillo. La declaración de uso de IA es un documento aparte, de una página como máximo
Trabajo	Por parejas

1. Objetivo

En la ciudad de RoboboSim hay un entorno llamado “Follow City” que contiene un Robobo, el **vehículo guía**, que circula continuamente por el anillo exterior con una pelota verde en su parte trasera. La tarea consiste en desarrollar un controlador mediante aprendizaje por refuerzo para que otro Robobo, el **seguidor**, aprenda a circular detrás de él: **manteniendo la separación dentro de la banda objetivo, sin chocar, sin perderlo de vista y sin salirse de su carril** (este último requisito con una tolerancia).

El seguidor se controla con la política obtenida mediante aprendizaje por refuerzo en un espacio de acciones continuo. El desarrollo se realiza en simulación, pero la política debe **ejecutarse después en el entorno real**, es decir, sobre la ciudad construida en el laboratorio 0.0 de la FIC, sin volver a entrenar.

2. El escenario “Follow City” de RoboboSim

- **El vehículo guía** circula solo por un carril del anillo exterior, girando siempre hacia la izquierda, y no reacciona al seguidor: si éste se detiene, el guía sigue su camino. Su conducción se configura en el simulador con un modo (continuo o variable) y unos límites de velocidad, entre 10 y 40 en las unidades del simulador.
- **Modos de conducción**: en **crucero** aplica la velocidad máxima en las rectas y la mínima en las curvas; en **variable** fluctúa entre ambas, frenando también en las curvas. Existe un tercer modo que añade paradas aleatorias, que **no se utiliza en esta práctica**: el guía nunca se detiene.
- **La pelota verde** es lo que permite seguir al guía. La detecta el módulo de blobs de color del propio robot. Puede valorarse además el uso de los infrarrojos para detectar al guía cuando está muy cerca.
- **El seguidor** es el robot al que se conecta el nodo puente de *robobo_ros2*. Todo el control y toda la percepción pasan por ROS 2.
- **En el entorno real** el vehículo guía es otro Robobo con la misma pelota, conducido por el profesorado o por un programa que recorre el anillo.

3. Herramientas

- **Gymnasium**: estructura del entorno de aprendizaje.

- **Stable-Baselines3**: implementación de los algoritmos de aprendizaje por refuerzo.
- **robobo_ros2**: nodo puente entre ROS 2 y el robot, simulado o real. Es la única vía de acceso a los sensores y actuadores del seguidor. Su interfaz está resumida en el anexo A.
- **El módulo sim del nodo puente**: da acceso por ROS 2 a la información del simulador, la posición y la orientación de un robot, y permite colocarlo y reiniciar la escena. No hace falta instalar ninguna biblioteca aparte. No existe en el entorno real.

No se proporciona código de partida. Todo el software de la práctica lo desarrolla cada grupo: el entorno de Gymnasium, la comunicación con ROS 2, las herramientas de medida del sensor, la colocación del seguidor en la escena y el evaluador del anexo B. *El uso de asistentes de IA para construirlo está permitido, pero su uso deberá ser explicado en un documento específico, y se realizará una defensa para valorar el grado de conocimiento del código entregado. El estudiante es el autor de la práctica, no el asistente de IA, por lo que debe ser capaz de modificarlo y arreglarlo sin ella.*

4. Espacio de observación y recompensa

> **El espacio de observación sólo puede contener información de los sensores del seguidor, recibida a través de ROS 2.** La posición del guía o del seguidor, la geometría del carril, la separación real o el error lateral son información privilegiada del simulador y no pueden formar parte de la observación.

La información privilegiada que ofrece el módulo sim, y cualquier magnitud que se derive de ella, sí puede emplearse para preparar el inicio de cada episodio, para decidir cuándo termina y para registrar métricas. El criterio es sencillo: **lo que la política necesita para decidir debe existir también en el robot real**; lo que sólo sirve para entrenar o para medir, no.

La recompensa sólo se usa durante el entrenamiento, así que podría calcularse con cualquier dato del simulador sin estorbar a la transferencia. Aun así, **se recomienda que se construya con magnitudes que el robot pueda inferir de su observación**, aunque sea de forma indirecta y con ruido. Este requisito no es obligatorio, pero si se da recompensa con datos que no están en el espacio de observación, **se deberá justificar en la memoria**.

La razón es que la red nunca ve la fórmula: sólo recibe un número por paso. Si ese número depende de algo que el agente no percibe, la misma observación con la misma acción cobra unas veces mucho y otras poco, y esa diferencia, para él, es ruido. El crítico aprende entonces un valor promediado sobre situaciones que no puede distinguir.

La separación, por ejemplo, es inferible: el tamaño del blob crece al acercarse. La posición respecto al carril no lo es, porque nada de lo que el robot percibe la delata; sirve como condición de fin de episodio y como métrica, pero no como término de la recompensa. Para cada término de la recompensa hay que decir en la memoria de qué magnitud depende y cómo puede inferirla el robot.

5. Desarrollo

5.1. Caracterización de los sensores

Se recomienda construir una herramienta de medida propia que permita determinar en qué rango de separaciones se detecta la pelota, cómo varían su posición y su tamaño en la imagen con la distancia, cuánto ruido tiene, en qué se diferencian las rectas de las curvas y cómo influye la inclinación de la cámara.

Ayuda para el uso de los blobs: el rango mínimo y máximo del detector del área del blob va de 1 a 1000.

En caso de utilizar otros sensores, como los IR, también se recomienda una caracterización similar.

5.2. Entrenamiento

En esta práctica se proponen **3 configuraciones del robot guía**, explicadas en el **Anexo B**. Cada una de ellas es de mayor dificultad para el aprendizaje, y la calificación final irá en proporción a cuál resuelva cada grupo. Para cada una de ellas, debéis entrenar una política que la resuelva de forma óptima, para lo cual debéis considerar:

- **Entorno Gymnasium:** cada grupo crea su propio entorno de Gymnasium que envuelve al seguidor. Hay que definir y justificar por escrito en la memoria (podrían variar entre configuraciones):
  - **Espacio de observación:** qué información recibe la política, de qué sensores procede, cómo se normaliza y qué ocurre cuando la pelota no se ve.
  - **Espacio de acción:** obligatoriamente continuo, con sus límites físicos justificados.
  - **Función de recompensa:** con cada término separado, el valor numérico de cada coeficiente y la magnitud observable de la que depende, según el apartado 4.
  - **Condiciones de terminación y truncamiento de los episodios.**
  - **Reinicio de los episodios:** cómo queda el escenario al empezar cada episodio durante el entrenamiento, y qué se hace mientras el guía sigue circulando.
- **Algoritmo**: se puede seleccionar cualquiera entre los de Stable-Baselines3 que admiten acciones continuas y se hayan visto en clase. Hay que justificar la elección teniendo en cuenta que cada paso del entorno cuesta tiempo real, **siendo obligatorio probar al menos SAC y PPO en cada configuración del Anexo B.**
  - **Configuración del guía**: durante el entrenamiento se puede usar cualquier modo y cualquier par de velocidades dentro del rango, pero para la evaluación se usarán las 3 del Anexo B.
- **Línea base**: se recomienda evaluar una política aleatoria con el protocolo del anexo B. Un entrenamiento que no la supere claramente no ha aprendido (esta parte es opcional, pero se valora en la calificación).

5.3. Evaluación con el protocolo común

El resultado final del entrenamiento se evalúa siguiendo el **protocolo del anexo B**: tres ejecuciones de 120 s con tres configuraciones del vehículo guía fijadas, iguales para todos los grupos, medidos con la verdad del terreno y con criterios de fallo propios del protocolo, con independencia de la recompensa de cada grupo. El evaluador lo implementa cada grupo, pero el protocolo es común y no admite variaciones: las métricas deben calcularse exactamente como se definen, para que los resultados sean comparables.

5.4. Transferencia al entorno real

Se deberá ejecutar la política en el entorno real **sin volver a entrenar**. Antes hay que repetir la caracterización del sensor sobre el robot real, porque los valores no coinciden con los del simulador. Conviene que todo lo que dependa de la calibración esté en un archivo de configuración y no en el código.

Hay que documentar qué se degrada y por qué. **Se evalúa el análisis, no el éxito**: una transferencia parcial bien explicada puntúa más que una que funciona sin explicación. Se valorarán las medidas tomadas durante el entrenamiento para hacer posible la transferencia.

6. Consideraciones y posibles errores

Los siguientes fenómenos pueden aparecer durante el desarrollo. Cómo tratarlos forma parte del trabajo, y la memoria debe mostrar cómo se han identificado y abordado.

- **El guía no espera.** Una política a medio entrenar lo pierde con frecuencia, y una vez perdido no reaparece. Además, su velocidad no es constante: frena en las curvas y acelera en las rectas, de modo que la separación no se mantiene sola.
- **Las curvas.** Seguir al vehículo guía no es lo mismo que seguir el camino por el que ha pasado. Se valorará buscar estrategias para que el robot seguidor esté dentro del carril derecho, si es posible
- **Frecuencias y retardos.** Las frecuencias del anexo A son las de publicación del nodo puente y están medidas en tiempo real, no en tiempo simulado: no cambian al acelerar el simulador, pero a mayor velocidad el robot recorre más distancia entre dos lecturas consecutivas. El periodo de control es una decisión de diseño y hay que declararlo. El temporizador de seguridad de cmd_vel, de 0,5 s, también cuenta tiempo real.
- **Un blob por color.** El detector entrega un único blob verde por imagen. En la ciudad real hay otros elementos verdes.
- **La velocidad del simulador.** A velocidad x4 cada paso mueve al robot cuatro veces más lejos y se percibe con cuatro veces menos muestras por metro recorrido: cambia la dinámica del problema, no sólo el tiempo de entrenamiento. Es una de las diferencias entre la simulación y el entorno real que hay que tener en cuenta.

7. Condiciones

- El espacio de acción debe ser continuo.
- El nodo puente se lanza sólo con los módulos necesarios. El de cámara no hace falta para detectar blobs.
- La interfaz de actuación es libre: el topic *cmd_vel*, los servicios de movimiento de ruedas o las acciones equivalentes. Hay que declarar cuál se usa y por qué.
- El multiplicador de velocidad del simulador se declara en la memoria y es el mismo en entrenamiento y en evaluación.
- Los parámetros que dependen de la calibración se guardan en un archivo de configuración.
- El protocolo de evaluación del anexo B se sigue sin modificaciones.

8. Elementos a entregar

Un archivo .zip con cuatro cosas:

8.1. Memoria

En formato PDF, de seis páginas como máximo más los anexos que hagan falta, en Calibri de 11 pt con interlineado sencillo: texto justificado, figuras y tablas numeradas con pie, valores numéricos explícitos en la función de recompensa y todos los resultados del apartado 8 comentados.

La estructura debe ser:

1. Definición del espacio de observación, espacio de acción, función de recompensa, condición de terminación y truncamiento, y reinicio de los episodios. Comentad aquí las decisiones de diseño que os parezcan más relevantes.
2. Pruebas realizadas para cada una de las 3 configuraciones del ANEXO B, incluyendo las curvas de aprendizaje de los entrenamientos, y tablas de datos si se consideran necesarias. Es importante probar con diferentes algoritmos, diferentes parámetros y realizar varias ejecuciones antes de seleccionar la mejor política para cada configuración.

3. Tablas de métricas del anexo B, para la línea base, y la solución final de cada configuración.
4. Conclusiones: análisis del resultado obtenido, indicando problemas y mejoras.

Curvas de aprendizaje

Una curva de aprendizaje muestra cómo evoluciona el rendimiento de la política a lo largo del entrenamiento. Sirve para comprobar que el agente aprende, para detectar cuándo deja de mejorar o se vuelve inestable, y para comparar configuraciones. Cada entrenamiento de la solución final se acompaña de curvas que cumplan los requisitos siguientes.

- **Magnitudes**: como mínimo, la recompensa acumulada por episodio y la duración del episodio, que son las que registra Stable-Baselines3 y muestra TensorBoard (rollout/ep_rew_mean y rollout/ep_len_mean). Se recomienda añadir la fracción de episodios que terminan por cada causa (colisión, pérdida del guía, límite de tiempo), porque una recompensa media que sube puede ocultar un cambio en el modo en que terminan los episodios.
- **Eje horizontal**: pasos del entorno, es decir, llamadas a step(). No se admiten episodios ni tiempo de reloj, porque la duración de los episodios cambia durante el entrenamiento y el tiempo depende del ordenador. Las curvas que se comparan en una misma figura comparten la escala de este eje.
- **Eje vertical**: nombre de la magnitud y su unidad. La recompensa es adimensional; la duración se expresa en pasos o en segundos, indicando la duración de un paso; las fracciones, en tanto por uno o en porcentaje.
- **Suavizado**: los valores por episodio pueden tener mucho ruido. En tal caso, se debe representar una media móvil, indicando el tamaño de la ventana en episodios, y detrás, con un trazo tenue, la serie sin suavizar.
- **Semillas**: la configuración final se entrena con al menos tres semillas distintas. La figura muestra la media y una banda de dispersión, indicando si corresponde al mínimo y el máximo o a la desviación típica.
- **Referencia**: una línea horizontal con la recompensa media por episodio que obtiene la política aleatoria en el propio entorno de entrenamiento.
- **Origen de los datos**: los valores se registran durante el entrenamiento, por ejemplo, con el envoltorio *Monitor* de *Stable-Baselines3*, que escribe un archivo CSV con una fila por episodio, o exportando el registro de TensorBoard. Las figuras de la memoria se generan con código propio a partir de esos datos, con una biblioteca de gráficos como Matplotlib o Seaborn. TensorBoard es una herramienta útil para seguir el entrenamiento mientras se ejecuta, pero sus capturas de pantalla no se admiten como figuras, porque no permiten combinar varias semillas en una banda, superponer la referencia ni fijar rótulos y unidades.
- **Presentación**: cada figura va numerada y con un pie que indica qué configuración se muestra, cuántas semillas, el tamaño de la ventana y el número total de pasos. Cada figura se comenta en el texto: a partir de qué momento la política supera a la referencia, si el rendimiento se estabiliza y si hay caídas, y a qué se atribuyen.

> Una curva de aprendizaje no sustituye a la evaluación. Durante el entrenamiento, la política actúa con exploración y se mide con la recompensa de cada grupo; el protocolo del anexo B usa la política determinista y cuatro métricas comunes a todos los grupos, que son las que miden la tarea. Una recompensa alta con episodios sin completar delata una recompensa mal ajustada.

8.2. Evidencias

Las figuras y las tablas de la memoria deben poder rehacerse a partir de lo que se entrega. Por eso se guardan y se entregan los registros que las generan: el registro por episodio de cada entrenamiento y el registro por paso de cada evaluación, en CSV o en el formato que produzca vuestro código, junto con los modelos y su configuración. No hay un formato obligatorio; lo que se exige es que estén y que el código que los lee venga con ellos.

Se deben añadir, además, videos de la ejecución final de cada política seleccionada en las tres configuraciones.

8.3. Declaración del uso de IA

Documento aparte, de una página como máximo, indicando qué herramienta/s de IA se han utilizado y para qué. Se debe dejar por escrito cuál ha sido la contribución propia, y cuál la de la IA.

8.4. Código fuente comentado

Se debe incluir el código fuente utilizado, con instrucciones para reproducir la ejecución de la política seleccionada como final en cada configuración.

Anexo A. Interfaz de ROS 2

Con los valores por defecto del nodo puente, el espacio de nombres es /robobo/robot_0: los recursos del robot cuelgan de base/ y smartphone/, y los del simulador de sim/. Los relevantes para esta práctica son:

Recurso	Tipo	Contenido
`smartphone/color_blobs`	`BlobArray, 5 Hz`	Lista de blobs: color, x, y, size. Se publica también vacía
`smartphone/set_active_color_blobs`	`servicio`	Colores a detectar: red, green, blue, custom (bool)
`base/cmd_vel`	`Twist`	linear.x en m/s y angular.z en rad/s. Se detiene si no recibe órdenes en 0,5 s
`base/move_wheels`	`servicio`	Velocidad de cada rueda, en unidades del robot
`base/move_wheels_time`	`servicio`	Lo mismo, durante un tiempo dado. También como acción
`base/move_tilt`	`servicio`	angle y speed de tipo float32
`base/move_pan`	`servicio`	angle y speed de tipo int32
`base/wheel/{left,right}/speed`	`Int32`	Velocidad medida de cada rueda

Para actuar se puede usar cmd_vel, en m/s y rad/s y con parada automática si deja de recibir órdenes, o los servicios de ruedas, en velocidades de rueda. La elección es libre.

```
ros2 run robobo_ros2 robobo_container --ros-args \
     -p ip:=host.docker.internal -p modules:="['blob','sim']"
```

El módulo sim da acceso al simulador sin instalar nada más. Trabaja en milímetros y grados, con el suelo en el plano (x, z):

Recurso	Tipo	Contenido
`sim/robot_location`	`RobotLocation`	id, position y rotation de ese robot
`sim/pose`	`Pose`	Lo mismo, con la orientación en cuaternión
`sim/set_robot_location`	`servicio`	Coloca ese robot: position y rotation
`sim/reset_simulation`	`servicio`	Reinicia la escena

Cada contenedor publica la posición de su propio robot, el de sus parámetros robot_name y robot_id. Para conocer también la del guía hace falta un segundo contenedor con el índice del otro robot y sólo el módulo sim.

Anexo B. Protocolo de evaluación

B.1. Magnitudes de referencia

Magnitud	Definición
Pelota verde	Referencia de todas las medidas, porque es lo que el seguidor percibe. Su posición sale de la pose del robot guía desplazada hacia su parte trasera
Separación	Distancia entre el centro del seguidor y la pelota verde
Banda objetivo	Separación entre 0,20 m y 0,45 m (en caso de usar otra, se debe justificar en la memoria)
Desviación	Ángulo entre el rumbo del seguidor y la dirección a la pelota. Corresponde a la x del blob, y comparar las dos, comprueba la calibración
Parámetros	Pelota 0,08 m por detrás del centro del guía · colisión por debajo de 0,12 m · guía perdido por encima de 0,90 m (en caso de usar otros, se deben justificar en la memoria)

NOTA: en la versión 4.3.0 de RoboboSim, es posible detectar la posición del robot guía, por lo que se podría usar esta posición y no la de la pelota verde para las medidas.

B.2. Condiciones de la evaluación

Cada episodio empieza con un reinicio de la simulación: ambos robots parten de donde los coloca el escenario y el seguidor parte parado, aunque se admiten ciertas acciones "de arranque" predeterminadas. Lo único que se fija es la configuración del guía, que nunca incluye paradas, y la política se ejecuta en modo determinista. Se deben analizar 3 ejecuciones de 120 segundos, con estos valores para el guía:

Configuración	Modo	Velocidad mínima	Velocidad máxima
C1. Lenta constante	Crucero	10	10
C2. Contraste	Crucero	10	40
C3. Fluctuante	Variable	10	40

B.3. Fin de cada episodio

Cada episodio dura 120 s y se da por completado si en ese tiempo no se produce ninguno de estos fallos, que se comprueban en cada paso:

- **Colisión**: separación menor que 0,12 m. (en caso de usar otra, se debe justificar en la memoria)
- **Guía perdido**: separación mayor que 0,90 m, o la pelota sin detectarse durante más de tres segundos seguidos (en caso de usar otra, se debe justificar en la memoria)

Una terminación del entorno propio por otra causa se registra como tal y no cuenta como completado. **Salirse del carril no es fallo del protocolo**, porque la geometría del carril no está disponible: se juzga en los vídeos.

B.4. Registro y métricas

En cada paso se registra el instante, la posición del seguidor y de la pelota (o del guía), la separación, la desviación lateral y la acción aplicada. De ahí salen cuatro métricas, en conjunto y desglosadas por configuración:

Métrica	Definición
Completados	Número y porcentaje de episodios completados
Desenlaces	Número de episodios terminados por cada una de las causas de B.3
Separación	Porcentaje de pasos dentro de la banda objetivo, y separación media
Desviación	Media del valor absoluto, en grados, y porcentaje de pasos por debajo de 15°